

586 Midterm Bananas for Bananas Forecasting

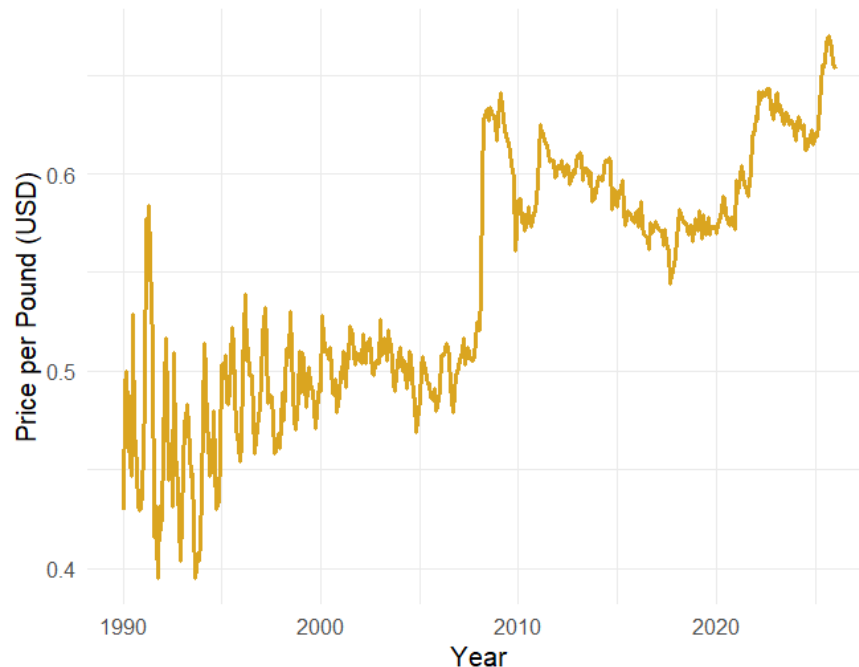
Quinlan Day

September 8, 2026

The recent project I was assigned was to simulate and predict a monthly forecast of banana prices from 01/01/2026 - 01/01/2027 using multiple models for comparison and their 95th and 5th percentile values. The overall data for the price of bananas since 1990 is below.

Historical Banana Prices (1980 - 2026)

Source: FRED (APU0000711311)

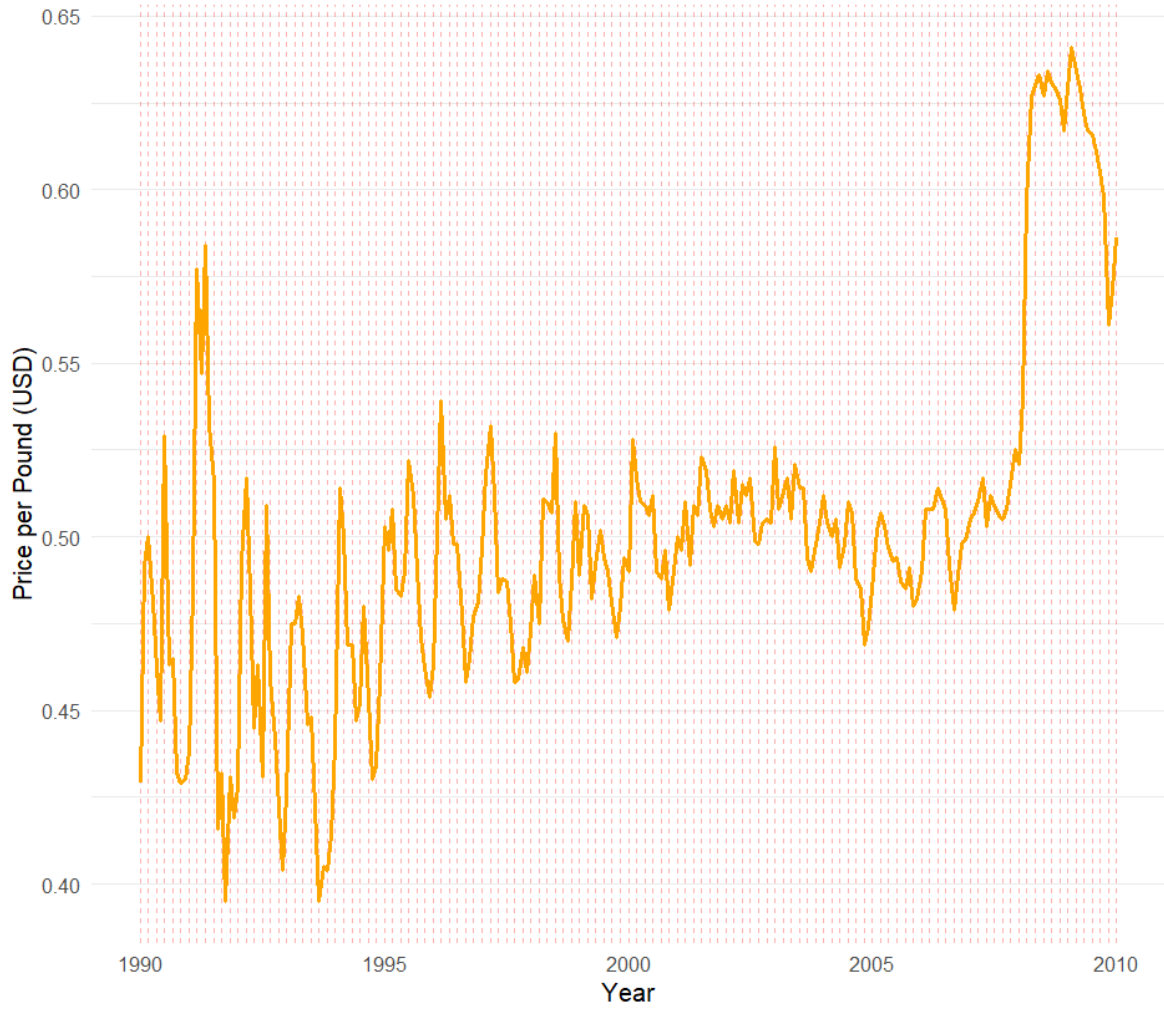


As you may notice, there is a large amount of volatility before 2000, which can be broken down into a smaller plot. Here, I have bimonthly red lines which seem to line up well with peaks and troughs, speaking to the seasonality of earlier data. Bananas have a history of being a loss leader, meaning its value is not in the profit it develops as an item, but rather the traffic it creates and the increased likelihood of people buying other items in the store. Because of this, we can see the high variation in the early 90's makes sense.

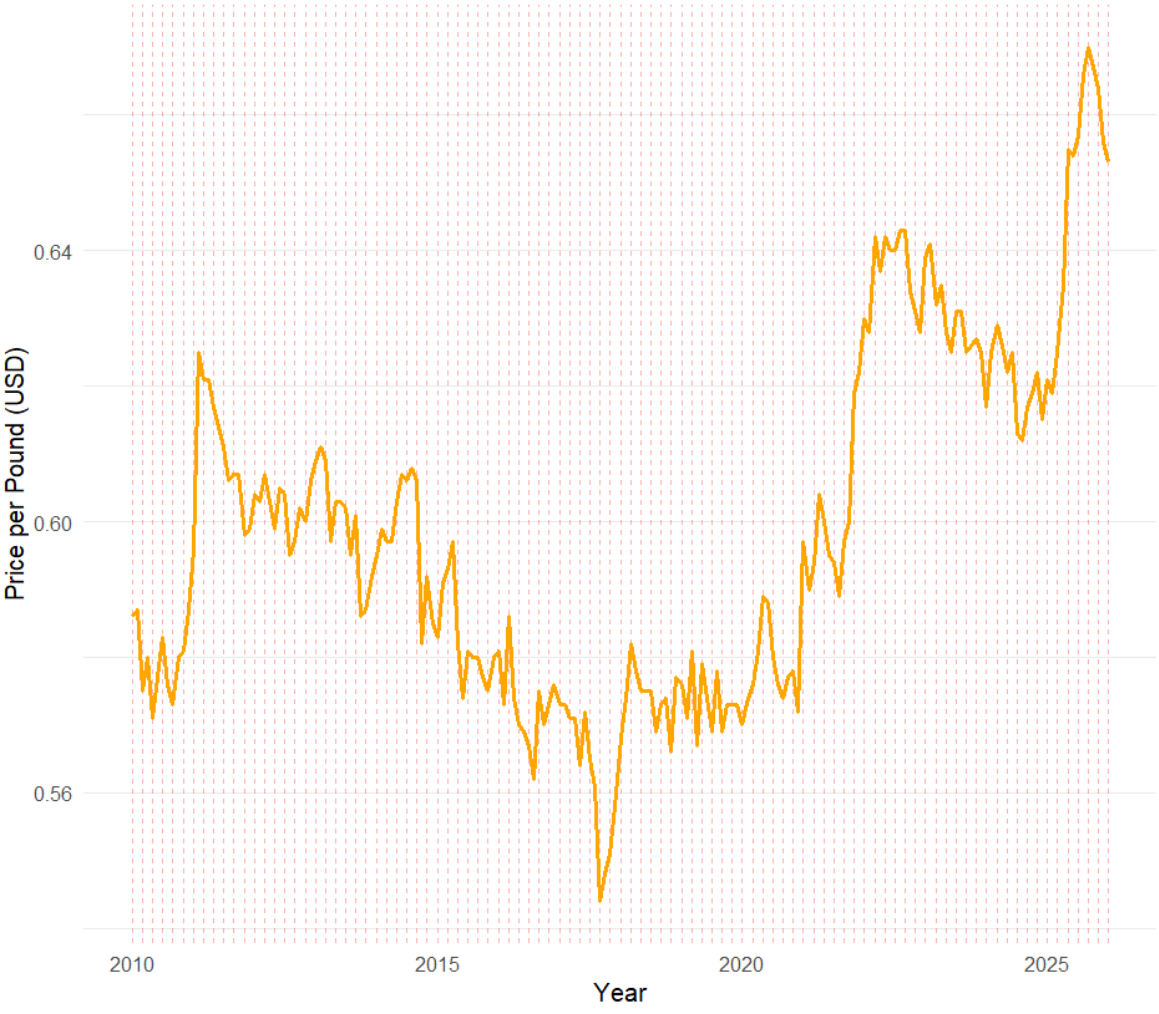
Below is a plot of the Banana price per lb from 1990-2010, which is about the end of the heavy volatility spike in the overall graph. This data is well broken up by two month red lines for the first two years, then becomes sparsely accurate past 1993. Later, there will be a discussion for the acf plot of the STL decomposition from 1990-2026 compared to 2010-2026. There is still heavy seasonality not stripped by the decomposition. However, if we take out the data from 1990-2010, the trend becomes very statistically insignificant.

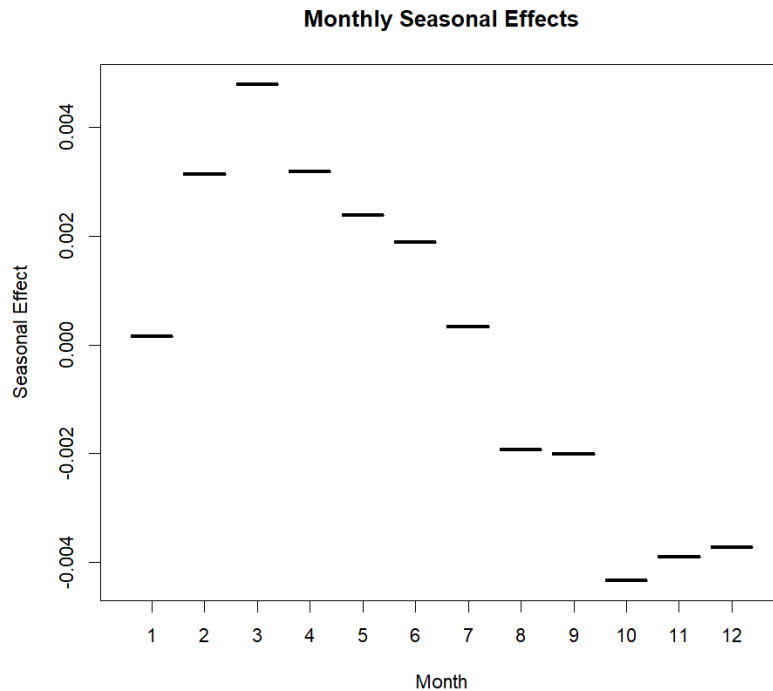
Another plot of the price after 2010 is below the first plot, which seems to continue to be well broken up by bimonthly increments, so this trend stays fairly consistent throughout the data.

Historical Banana Prices: Bimonthly Grid



Historical Banana Prices: Bimonthly Grid





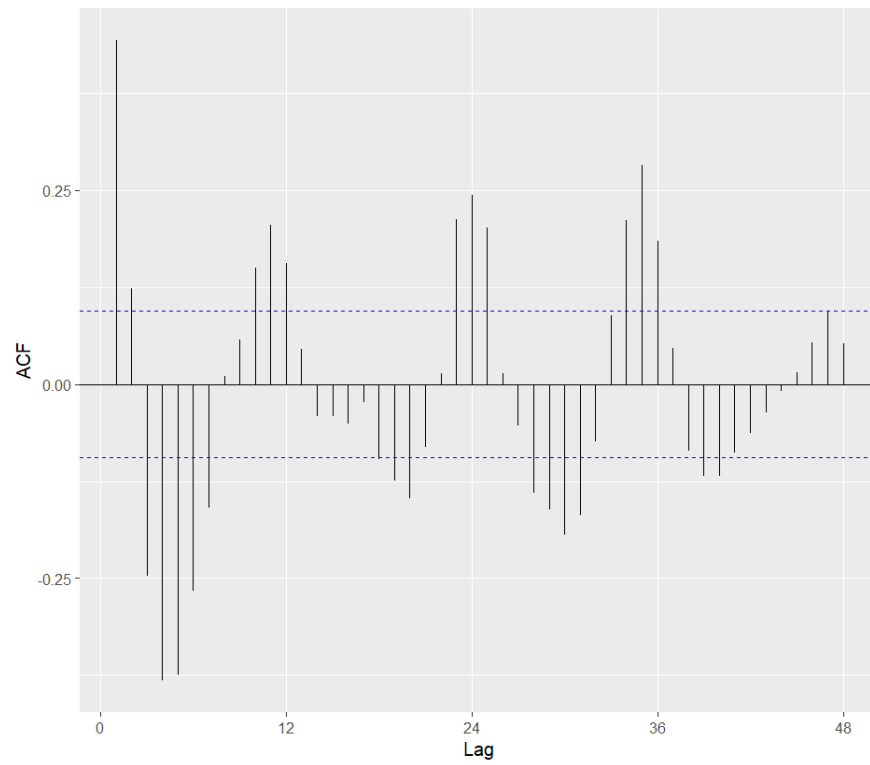
Overall the raw data is:

- Broken up well by bimonthly segments.
- Extremely volatile before 2000 and does not seem to follow the same trend before and after 2010.
- Is missing October 2025, which has been replaced with the value 0.667.
- Unique in the fact that it is a loss leader and does not follow a typical price distribution.
- Seasonally it increases from January into February, then dips into its lowest in October and makes its way back up.

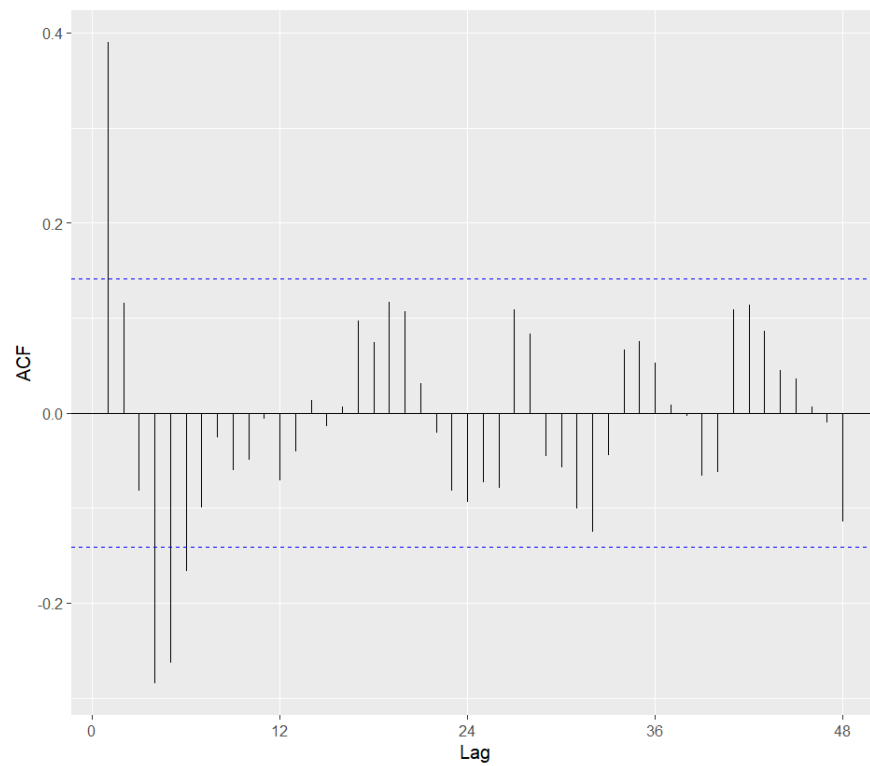
An STL decomposition also speaks to the validity of removing the data from 1990-2010. There is a large wave pattern in the remainder data from 1990-2000, and using data from before 2010 leads to heavy violations of our normality assumption. By removing the data from 1990-2010, our Breusch-Pagan test goes from $8.727 \cdot 10^{-13}$ to 0.0531. Using a log transform does not make the variance constant either as its p-value goes to $7.932 \cdot 10^{-15}$. Removing the odd data from 1990-2010 provides a constant variance assumption, which is why we will be continuing with only data from 2010-2026. Since we are dealing with monthly data, having 36 years of historical data is nice, but can tend to provide less accurate analysis than using strictly newer data, especially in instances like this. Stationarity statistics will be given later on, but leaving in data from before 2010 violates the 24 and 36 month stationarity assumptions, further validating the choice to use only a portion of the data. When finding a Box-Cox value, the R code function returns a transform of -1 which would make the model extremely hard to interpret since it flips price data so that large values become small and small values become large. Restricting our Box-Cox to a min of 0 and max of 1, we obtain a value of 1, and since we do not violate any stationarity or linear assumptions we can continue with our data as is.

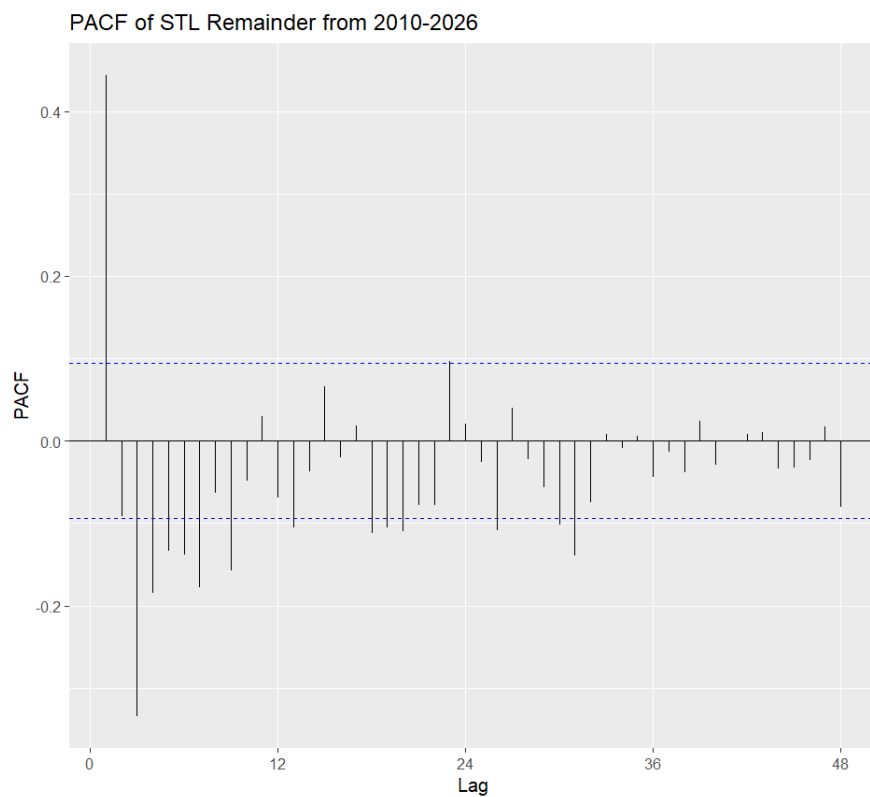
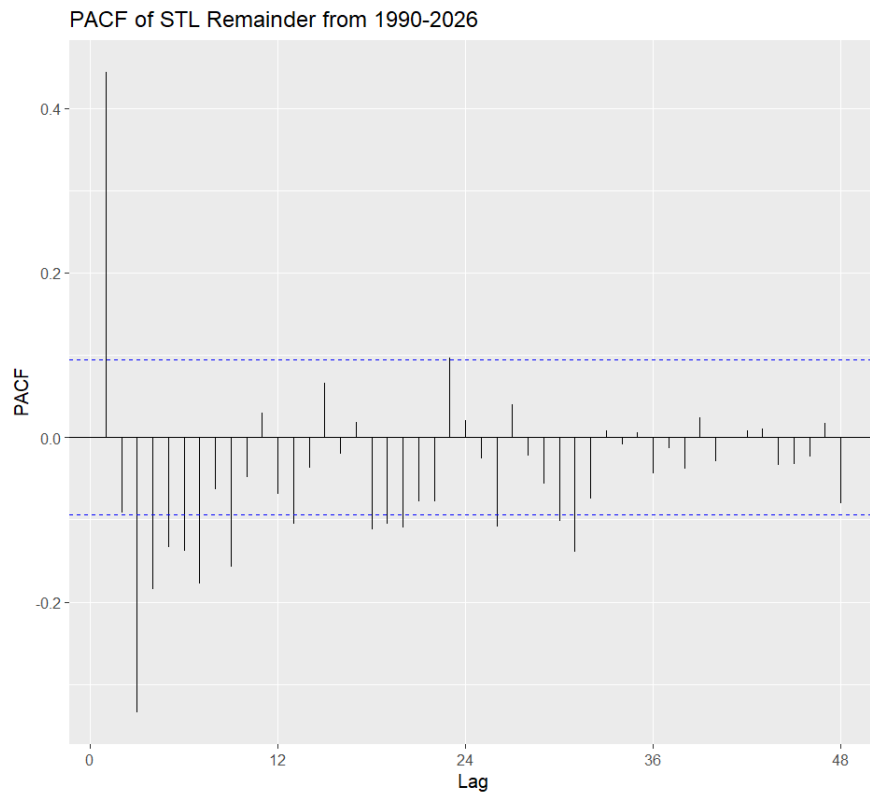
Here are two plots for the acf of the STL remainder from 1990-2026 and below it the remainder from 2010-2026. The graph from 2010-2026 is much much easier to work with and will likely give better data, considering the STL decomposition removed more of the seasonality than it did for the full data. Noticeably, we still need to difference the data, which is the reason for the large spike in the beginning of both the ACF and PACF graphs. The PACF graph also benefits highly from removing early data, showing much less significant spikes in the diagram.

ACF of STL Remainder from 1990-2026



ACF of STL Remainder from 2010-2026





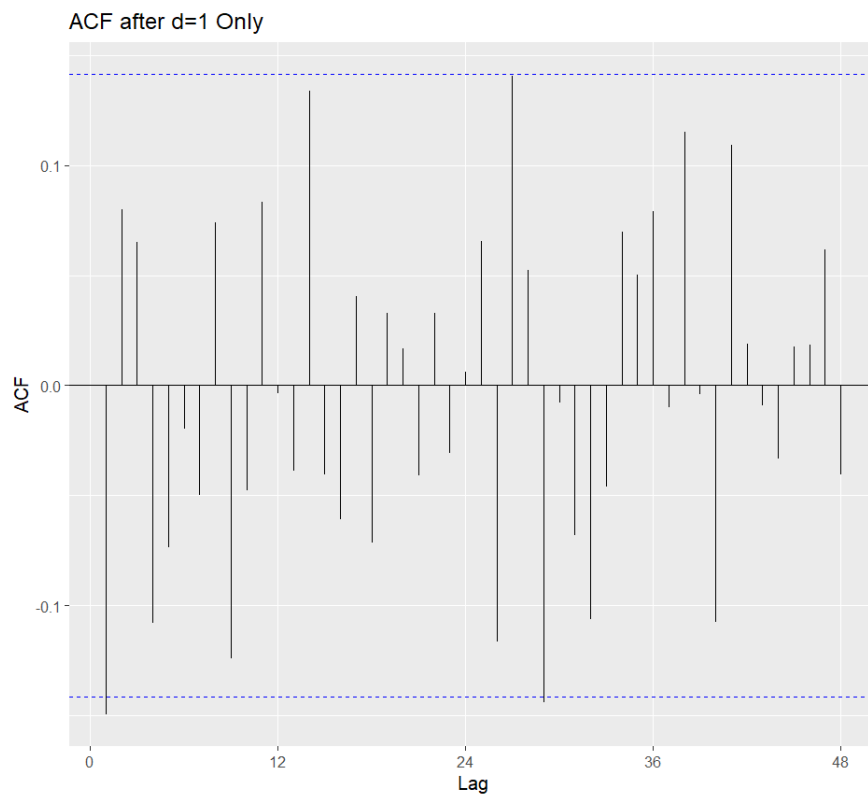
After we perform our differencing, the data becomes much more similar to white noise with a few abnormal lags. The plots after differencing ($d=1$, $D=1$) are below. The untransformed data fails both the Augmented Dickey-Fuller (ADF) test and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) tests. A

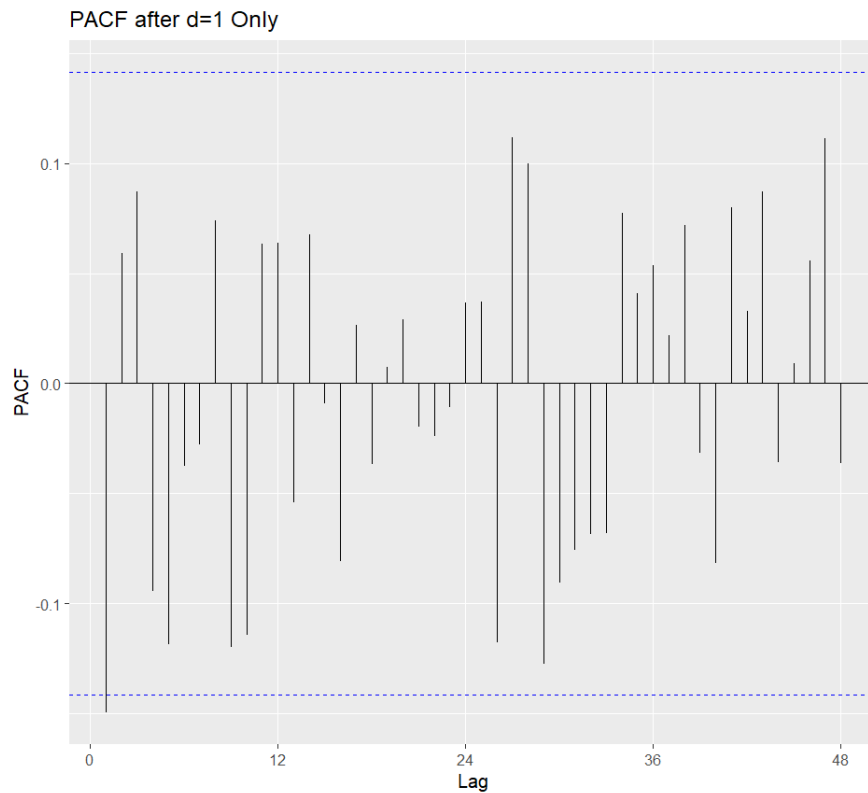
table of its p-values are below:

Table 1: Unit Root and Stationarity Diagnostics for Banana Price Series

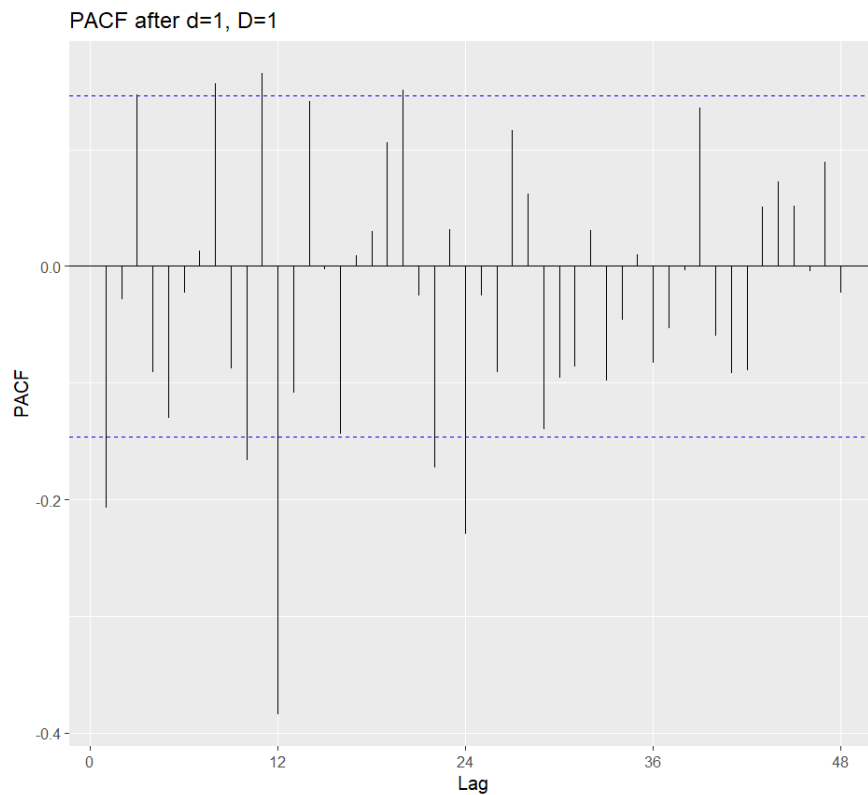
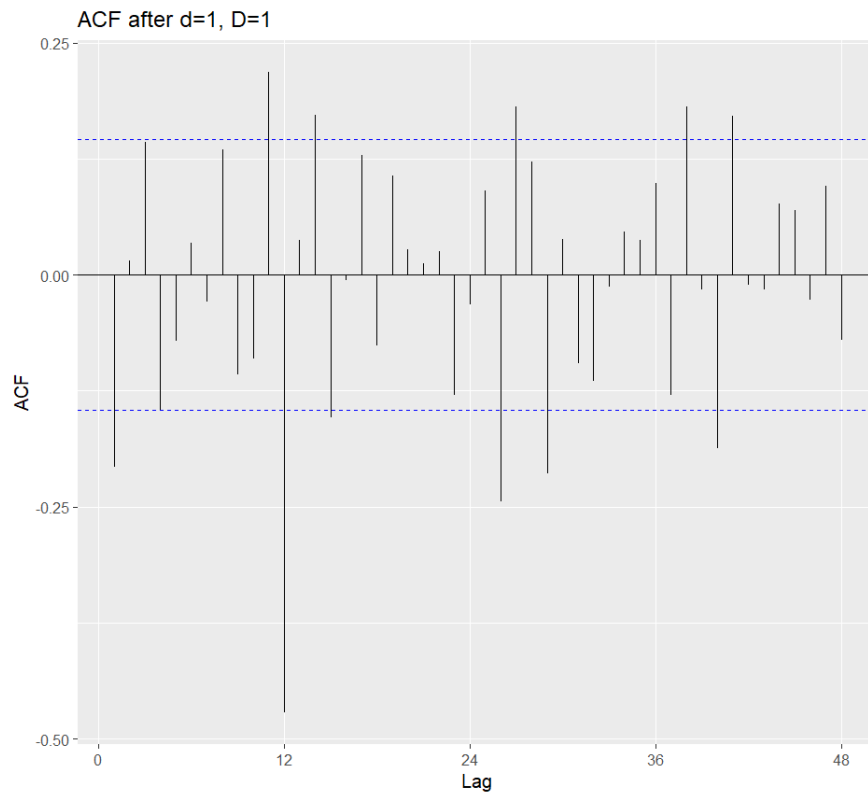
Transformation Stage	ADF Test (Unit Root)		KPSS Test (Stationary)		Status
	<i>p-value</i>	<i>Result</i>	<i>p-value</i>	<i>Result</i>	
Raw Transformed Series	0.8416	Non-Stationary	≤ 0.0100	Non-Stationary	Diff Needed
After $d = 1$ Only	≤ 0.0100	Stationary	≥ 0.1000	Stationary	Stationary
After $D = 1$ Only (lag = 12)	0.0696	Non-Stationary	0.0547	Stationary	Diff Needed
After $d = 1$ AND $D = 1$	≤ 0.0100	Stationary	≥ 0.1000	Stationary	Stationary

Given the price of bananas is only stationary for a $d = 1$ lag and both $d = 1$ and $D = 1$, we can continue the analysis. The ACF plot below for $d = 1$ alone gives an extremely nice looking ACF diagram, and that also applies to its PACF below it.





The ACF and PACF for the $d = 1, D = 1$ differencing is below. As you can see, using $d = 1$ almost entirely gets rid of all significant lags, except for at lag 1, which means our seasonality is not present enough to be noticeable until we use the $D = 1$ term, then we can capture that seasonality with a Q and/or P value of 1, so our solution will have to use a $D = 1$ seasonal term to capture the seasonality correctly. The PACF graph shows the same, that not using D gives no significant autocorrelations other than lag 1, which barely passes the significance threshold, so we can limit p to 1 or 0. The `auto.arima` model that results from purely $d = 1$ is a $p = 2, q = 2$ model which ends up giving the AR and MA coefficients opposite values close in magnitude, meaning the model is overfitting. Preventing p and q from being both equal to 2 returns the `auto.arima` for an `AR(1)` process.



Using a range of arima functions we can see that there is a tradeoff between RMSE and MASE. Having $d = 1, D = 0$ results in better RMSE, while $d = 1, D = 1$ has better MASE. The best model we can use for $d = 1, D = 1$ while ranking MASE is the SARIMA model (0,1,2) (0,1,1). Using an AR term

would mean a price shock echoes and dies out over a long period of time, whereas no autoregressive term means the price shock will not last, a fitting conclusion for a loss leader. The other model that has the second lowest MASE is (1,1,2) (0,1,1), and since there was a lag spike at the first instance in the acf graph of $d = 1, D = 1$ we will use this for our $d = 1, D = 1$ model. This fits well with our note earlier about the prices being well broken up bimonthly, as well as the spike at the 12th lag in the $d = 1, D = 1$ acf plot which indicate our Q value of 1. Since we are interested in the monthly breakdown of the banana price per lb, this ARIMA process is our preferred method.

For a no seasonal ARIMA model, $d = 1$ and $D = P = Q = 0$. Looking at model comparisons, we can use RMSE as a measure of error, and MASE as a measure of trend capture. While RMSE is a measure of overall error, MASE is more apt towards this purpose due to its purpose of measuring trend capturing. Also, since the ACF and PACF plots did not have significant seasonal trends at $d = 1$ it is not wise to force seasonality, however, we can use the previous model mentioned as a comparison. Generating a number of arima models with $p < 1, d = 1, q < 2$ we obtain the following table:

Model	RMSE	MASE	AICc
ARIMA(1,1,2)	0.006892	0.347523	-1357.164
ARIMA(0,1,2)	0.006896	0.346917	-1359.024
ARIMA(1,1,1)	0.006927	0.345308	-1357.376
ARIMA(1,1,0)	0.006932	0.344661	-1359.126
ARIMA(0,1,1)	0.006943	0.344591	-1358.547
ARIMA(0,1,0)	0.007008	0.350573	-1357.012

Table 2: Model performance comparison for $d = 1, D = 0$ configurations.

Overall, selecting model 2 seems like a good balance. It has only a very slightly worse RMSE than the top candidate, but a fairly better MASE and has the second highest AICc. Moving forward, the two models we will be forecasting will be ARIMA(0,1,2) and SARIMA(1,1,2)(0,1,1).

Comparing these two models with the given benchmarks (Mean Model, Naive Model, Random Walk/-Drift Model, Seasonal Naive Model and an AR(1) model) using RMSE and MASE obtains the below table.

Table 3: Model Performance Metrics Comparison

Model	RMSE	MASE
AR(1)	0.006986	0.349474
Mean Model	0.026276	1.453674
Naive Model	0.007026	0.352255
Random Walk w/ Drift	0.007018	0.353954
Seasonal Naive Model	0.020219	1.000000
Auto ARIMA (0,1,2)	0.006896	0.346917
Auto SARIMA (1,1,2)(0,1,1)	0.006923	0.339635

Our models beat all benchmarks in RMSE, while MASE metric ended with the seasonal ARIMA model taking first best, AR(1) taking second best, and the (0,1,2) model taking third best. ARIMA(0,1,2) is the clear winner if the goal was overall accuracy towards the mean value of bananas, while SARIMA(1,1,2)(0,1,1) has better accuracy towards the individual monthly forecasts and captures the seasonal variant better while suffering slightly in overall accuracy. The final forecast is in the appendix, however since the ARIMA process is not seasonal, its mean will be a point estimate.

Appendix

Month	Mean			Naive			Drift			SNaive		
	Mean	Low	High	Mean	Low	High	Mean	Low	High	Mean	Low	High
January	0.598	0.546	0.650	0.653	0.639	0.667	0.653	0.640	0.667	0.619	0.579	0.659
February	0.598	0.546	0.650	0.653	0.634	0.672	0.654	0.634	0.673	0.625	0.585	0.665
March	0.598	0.546	0.650	0.653	0.629	0.677	0.654	0.630	0.678	0.635	0.595	0.675
April	0.598	0.546	0.650	0.653	0.625	0.681	0.654	0.627	0.682	0.655	0.615	0.695
May	0.598	0.546	0.650	0.653	0.622	0.684	0.655	0.624	0.686	0.654	0.614	0.694
June	0.598	0.546	0.650	0.653	0.619	0.687	0.655	0.621	0.689	0.657	0.617	0.697
July	0.598	0.546	0.650	0.653	0.617	0.689	0.655	0.618	0.693	0.666	0.626	0.706
August	0.598	0.546	0.650	0.653	0.614	0.692	0.656	0.616	0.696	0.670	0.630	0.710
September	0.598	0.546	0.650	0.653	0.612	0.694	0.656	0.614	0.698	0.667	0.627	0.707
October	0.598	0.546	0.650	0.653	0.609	0.697	0.656	0.612	0.701	0.664	0.624	0.704
November	0.598	0.546	0.650	0.653	0.607	0.699	0.657	0.610	0.704	0.656	0.616	0.696
December	0.598	0.546	0.650	0.653	0.605	0.701	0.657	0.608	0.706	0.653	0.613	0.693

Table 4: Forecast results for Mean, Naive, Drift, and SNaive

Month	AR1			ARIMA			SARIMA		
	Mean	Low	High	Mean	Low	High	Mean	Low	High
January	0.651	0.635	0.668	0.652	0.639	0.666	0.655	0.641	0.669
February	0.649	0.627	0.673	0.652	0.634	0.670	0.657	0.639	0.675
March	0.648	0.621	0.676	0.652	0.630	0.674	0.657	0.634	0.679
April	0.646	0.617	0.678	0.652	0.626	0.678	0.658	0.632	0.684
May	0.644	0.612	0.680	0.652	0.623	0.681	0.658	0.628	0.687
June	0.643	0.609	0.681	0.652	0.620	0.684	0.656	0.624	0.688
July	0.641	0.605	0.682	0.652	0.617	0.687	0.655	0.621	0.690
August	0.640	0.602	0.683	0.652	0.614	0.689	0.656	0.619	0.693
September	0.639	0.599	0.683	0.652	0.612	0.692	0.655	0.615	0.694
October	0.637	0.597	0.684	0.652	0.610	0.694	0.656	0.614	0.698
November	0.636	0.595	0.684	0.652	0.608	0.696	0.655	0.611	0.699
December	0.635	0.592	0.684	0.652	0.606	0.698	0.659	0.613	0.705

Table 5: Forecast results for AR1, ARIMA, and SARIMA

Code

```
library(quantmod)
library(ggplot2)
library(forecast)
library(zoo)
library(gridExtra)
library(tseries)
library(lmtest)

getSymbols("APU0000711211", src = "FRED")

bananas <- data.frame(
  Date = zoo::index(APU0000711211),
  Price = as.numeric(APU0000711211)
)

bananas <- subset(bananas, Date >= "2010-01-01" & Date <= "2026-01-01")
bananas$Price[is.na(bananas$Price)] <- 0.667

start`year` <- as.integer(format(min(bananas$Date), "%Y"))
start`month` <- as.integer(format(min(bananas$Date), "%m"))
bananas`ts` <- ts(bananas$Price, start = c(start`year`, start`month`), frequency = 12)

banana`df` <- data.frame(
  Date = as.numeric(time(bananas`ts`)),
  Price = as.numeric(bananas`ts`)
)

bimonthly`breaks` <- seq(
```

```

    from = min(banana$df$Date),
    to = max(banana$df$Date),
    by = 2/12
  )

  ggplot(banana$df, aes(x = Date, y = Price)) +
    geom_vline(xintercept = bimonthly$breaks, color = "red", linetype = "dashed", alpha = 0.3)
    +
    geom_line(color = "orange", linewidth = 0.9) +
    labs(
      title = "Historical Banana Prices: Bimonthly Grid",
      x = "Year",
      y = "Price per Pound (USD)"
    ) +
    theme_minimal() +
    theme(
      panel.grid.minor.x = element_blank(),
      panel.grid.major.x = element_blank()
    )
  )

  profile.distribution <- function(x, label) {
    x <- as.numeric(x)
    n <- length(x)
    sw <- shapiro.test(x)
    cat(sprintf(" %40s n=%d\n", label, n))
    cat(sprintf("      Mean=%.4f SD=%.4f\n",
                mean(x), sd(x)))
    cat(sprintf("      Shapiro-Wilk W=%.5f p=%.6f -> %s\n",
                sw$statistic, sw$p.value,
                ifelse(sw$p.value < 0.05, "NORMAL",
                      ifelse(sw$statistic < 0.97, "minor tails (W<0.97, practically normal)",
                            ifelse(sw$statistic < 0.95, "mild tails (W<0.95, acceptable)",
                                  "non-normal (W<0.95, investigate)")))))
  }

  ~

  # Raw prices
  prof$raw <- profile.distribution(bananas$ts, "Raw prices (bananas$ts)")
  # Log prices
  prof$log <- profile.distribution(log(bananas$ts), "Log prices log(bananas$ts)")
  # First differences of log (log returns)
  prof$ret <- profile.distribution(diff(log(bananas$ts)), "Log returns diff(log(bananas$ts))")

  # STL remainder of raw prices
  stl$raw <- stl(bananas$ts, s.window = "periodic")
  prof$rem <- profile.distribution(stl$raw$time.series[, "remainder"],
                                "STL remainder of raw prices")

  # STL remainder of log prices
  stl$log <- stl(log(bananas$ts), s.window = "periodic")
  prof$remlog <- profile.distribution(stl$log$time.series[, "remainder"],
                                    "STL remainder of log prices")

  lambda <- BoxCox.lambda(bananas$ts)
  print(lambda)
  lambda <- 1
  bananas$bc <- BoxCox(bananas$ts, lambda)

  p$full <- ggplot(bananas, aes(x=Date, y=Price)) +
    geom_line(color="goldenrod", linewidth=0.8) +
    labs(title = "STEP 4: Historical Banana Prices",
         subtitle = "Source: FRED (APU0000711211)",
         x="Year", y="Price per Pound (USD)") +
    theme_minimal()
  print(p$full)

  bananas$stl <- stl(bananas$bc, s.window = "periodic")

  # Extract the seasonal component as a time series
  seasonal$ts <- bananas$stl$time.series[, "seasonal"]
  # For a monthly series (frequency = 12)

```

```

seasonal`cycle` j- ts(seasonal`ts, frequency = frequency(seasonal`ts))
boxplot(seasonal`cycle ~ cycle(seasonal`cycle),
        main = "Monthly Seasonal Effects",
        xlab = "Month", ylab = "Seasonal Effect")
plot(seasonal`ts, ylab = "Seasonal Effect", main = "STL Seasonal Breakdown")

p`stl` j- autoplot(bananas`stl) +
  labs(title = "STEP 5: STL Decomposition")
print(p`stl)

remainder`stl` j- bananas`stl$time.series[, "remainder"]

print(ggAcf(remainder`stl, lag.max=48) +
      labs(title = "ACF of STL Remainder from 2010-2026"))

print(ggPacf(remainder`stl, lag.max=48) +
      labs(title = "PACF of STL Remainder from 1990-2026"))

run`stationarity` j- function(series, label) -
  adf`r` j- adf.test(series)
  kpss`r` j- kpss.test(series, null="Level")

  adf`p`exact` j- as.numeric(adf`r`$p.value)
  kpss`p`exact` j- as.numeric(kpss`r`$p.value)

  cat(sprintf("ADF p-value = %.6f -¿ %s`n", adf`p`exact,
              ifelse(adf`p`exact > 0.05, "NON-STATIONARY (difference needed)", "STATIONARY")
              ))
  cat(sprintf("KPSS p-value = %.6f -¿ %s`n", kpss`p`exact,
              ifelse(kpss`p`exact > 0.05, "NON-STATIONARY (difference needed)", "STATIONARY")
              )))

  invisible(list(adf=adf`r`, kpss=kpss`r`))
"

bananas`diff1` j- diff(bananas`bc, differences=1)
bananas`diff`annual` j- diff(bananas`bc, lag=12, differences=1)
bananas`diff12` j- diff(bananas`diff1, lag=12, differences=1)

run`stationarity(bananas`bc, "Raw transformed series")
run`stationarity(bananas`diff1, "After d=1 only")
run`stationarity(bananas`diff`annual`, "After D=1 only (lag=12)")
run`stationarity(bananas`diff12, "After d=1 AND D=1 j- target")

print(ggAcf(bananas`diff`annual`, lag.max=48) +
      labs(title="ACF after Annual Diff Only (D=1)"))

print(ggAcf(bananas`diff1, lag.max=48) +
      labs(title="ACF after d=1 Only"))

print(ggPacf(bananas`diff1, lag.max=48) +
      labs(title="PACF after d=1 Only"))

print(ggAcf(bananas`diff12, lag.max=48) +
      labs(title="ACF after d=1, D=1"))

print(ggPacf(bananas`diff12, lag.max=48) +
      labs(title="PACF after d=1, D=1"))

fit`mean` j- meanf(bananas`ts, h=24)
fit`naive` j- naive(bananas`ts, h=24)
fit`drift` j- rwf(bananas`ts, h=24, drift=TRUE)
fit`snaive` j- snaive(bananas`ts, h=24)
fit`ar1` j- Arima(bananas`ts, order=c(1,0,0), lambda=lambda)

if (TRUE) -

  results`list` j- list()
  counter` j- 1

  p`range` j- 0:1

```

```

d`range j- 1
q`range j- 0:2
P`range j- 0:1
D`range j- 0:1
Q`range j- 0:1

total`iters j- length(p`range) * length(d`range) * length(q`range) *
  length(P`range) * length(D`range) * length(Q`range)

cat(sprintf("%d combinations", total`iters))
for (p in p`range) -
  for (d in d`range) -
    for (q in q`range) -
      for (P in P`range) -
        for (D in D`range) -
          for (Q in Q`range) -

            model`label j- sprintf("SARIMA(%d,%d,%d)(%d,%d,%d)", p, d, q, P, D, Q)

            fit j- tryCatch(-
              Arima(bananas`ts,
                order = c(p, d, q),
                seasonal = list(order = c(P, D, Q), period = 12),
                lambda = lambda)
            ~, error = function(e) - return(NULL) ~)

            if (!is.null(fit)) -
              acc`metrics j- accuracy(fit)

              resids j- as.numeric(residuals(fit))
              resids j- resids[!is.na(resids)]
              sw`metric j- tryCatch(- shapiro.test(resids) ~, error = function(e) - list(
                statistic=0, p.value=0) ~)

              results`list[[counter]] j- data.frame(
                Model = model`label,
                RMSE = acc`metrics[1, "RMSE"],
                MASE = acc`metrics[1, "MASE"],
                AICc = fit$aicc,
                Shapiro`p = sw`metric$p.value,
                stringsAsFactors = FALSE
              )
              counter j- counter + 1

            "
          "
        "
      "
    "
  "
"

grid`results j- do.call(rbind, results`list)
grid`results j- grid`results[order(grid`results$MASE), ]

rownames(grid`results) j- NULL
print(head(grid`results, 15))
"

fit`temp j- auto.arima(bananas`ts, max.p = 1, d=1, max.D=1, max.q=2, max.P=1,max.Q=1)
fit`temp
autoplot(fit`temp)
fit`target j- Arima(bananas`ts, order=c(0,1,2), seasonal = list(order = c(0,0,0), period=12),
  , lambda = lambda)
fit`alt`Q j- Arima(bananas`ts, order=c(1,1,2), seasonal = list(order = c(0,1,1), period=12),
  lambda = lambda)

extract`diagnostics j- function(model`obj, label, fit`df) -
  if (is.null(model`obj)) return(NULL)

```

```

acc <- accuracy(model`obj)[1, ]
lb <- Box.test(residuals(model`obj), lag = 36, type = "Ljung-Box", fitdf = fit`df)

data.frame(
  Model = label,
  LjungBox`Stat = lb$statistic,
  LjungBox`p = lb$p.value,
  RMSE = acc["RMSE"],
  MASE = acc["MASE"],
  stringsAsFactors = FALSE
)

diagnostic`grid <- do.call(rbind, list(
  extract`diagnostics(fit`target, "target", fit`df = 2),
  extract`diagnostics(fit`alt`Q, "alt`Q", fit`df = 2)
))

rownames(diagnostic`grid) <- NULL
print(diagnostic`grid)

fit`norm`winner <- fit`target
print(fit`auto)
print(fit`target)

get`in`sample`metrics <- function(model`obj, label) -
  if (is.null(model`obj)) return(NULL)

  acc <- accuracy(model`obj)
  if (nrow(acc) < 1) -
    metrics <- acc["Training set", ]
  else -
    metrics <- acc[1, ]

  data.frame(
    Model = label,
    RMSE = metrics["RMSE"],
    MASE = metrics["MASE"],
    stringsAsFactors = FALSE
  )

performance`matrix <- do.call(rbind, list(
  get`in`sample`metrics(fit`ar1, "AR(1)"),
  get`in`sample`metrics(fit`mean, "Mean Model"),
  get`in`sample`metrics(fit`naive, "Naive Model"),
  get`in`sample`metrics(fit`drift, "Random Walk w/ Drift"),
  get`in`sample`metrics(fit`snaive, "Seasonal Naive Model"),
  get`in`sample`metrics(fit`target, "0 1 2"),
  get`in`sample`metrics(fit`alt`Q, "0 1 2 0 1 1")
))

rownames(performance`matrix) <- NULL
print(performance`matrix)

best`model`row <- performance`matrix[order(performance`matrix$RMSE), ][1, ]
best`name <- best`model`row$Model

best`fit <- switch(best`name,
  "Mean Model" = fit`mean,
  "Naive Model" = fit`naive,
  "Random Walk w/ Drift" = fit`drift,
  "Seasonal Naive Model" = fit`snaive,
  "AR(1) Model Structural" = fit`target,
  "SARIMA(1,1,1)(0,0,1)[12]" = fit`alt`Q,
  "Auto-ARIMA Optimized Matrix" = fit`auto)

```

```

fit`target
fit`alt`Q

fc`mean  <- forecast(fit`mean, h=12)
fc`naive <- forecast(fit`naive, h=12)
fc`drift <- forecast(fit`drift, h=12)
fc`snaive <- forecast(fit`snaive, h=12)
fc`target <- forecast(fit`target, h=12)
fc`alt`Q <- forecast(fit`alt`Q, h=12)
fc`ar1 <- forecast(fit`ar1, h=12)

forecast`list <- list(
  Target = fc`target,
  AltQ = fc`alt`Q,
  Mean = fc`mean,
  Naive = fc`naive,
  AR1 = fc`ar1,
  SNaive = fc`snaive,
  Drift = fc`drift
)

all`forecasts <- do.call(rbind, mapply(function(fc`obj, name) {
  data.frame(
    Month = 1:12,
    Model = name,
    Mean = round(as.numeric(fc`obj$mean), 3),
    Low = round(as.numeric(fc`obj$lower[, 2]), 3),
    High = round(as.numeric(fc`obj$upper[, 2]), 3)
  )
}, forecast`list, names(forecast`list), SIMPLIFY = FALSE))

print(all`forecasts)

```